DreamsPOS ERP

Documentation Technique — Backend Django REST API

Django 5.1.4 DRF 3.15.2 JWT + 2FA Multi-tenant i18n fr/en/ar OpenAPI / Swagger Campay MoMo

Version : 2.0.0 | **Date :** Juin 2026 | **Auteur :** Équipe DreamsPOS | **Statut :** Transport & Logistique — Complet

📋 Table des matières

1. Vision du projet
2. Stack technique
3. Architecture backend
4. Structure du projet
5. Module Authentication
6. Module System Config
7. Module Admin RBAC
8. Module Users
9. Module RH (Human Resources)
10. Module Fleet (Gestion de flotte)
11. Module Transport Config
12. Module Voyage (Gestion des départs)
13. Module Ticketing (Réservations & Billets)
14. Module Parcel (Colis)
15. Module Treasury (Trésorerie)
16. Module Online Booking (API Publique)
17. Récapitulatif des endpoints
18. Roadmap des modules
19. Conventions de développement

1. Vision du projet

DreamsPOS ERP est un système de gestion intégré (ERP) multi-tenant développé avec **Django REST Framework** et **Angular**. La première verticale implémentée est le secteur **Transport & Logistique** (modèle Blue Bird Express / Brainstore ERP).

VERTICALE TRANSPORT (V2.0)

- ✅ Gestion de flotte (véhicules, conducteurs, maintenance)
- ✅ Configuration transport (villes, terminaux, routes, tarifs)
- ✅ Gestion des voyages (départs, embarquement, équipage)
- ✅ Billetterie (réservations, billets QR, bagages)
- ✅ Colis (tracking REGISTERED→DELIVERED, paiement COD)
- ✅ Trésorerie (caisses, banques, sessions, virements)
- ✅ Réservation en ligne (API publique + Campay MoMo)

FONDATION ERP (V1.0)

- ✅ Authentification JWT + 2FA TOTP
- ✅ Multi-tenant (Company + Agencies)
- ✅ RBAC (Rôles, Profils, Permissions)
- ✅ Ressources Humaines
- 🔧 Finance OHADA, Inventaire, POS, CRM

RÉPERTOIRES DU PROJET

BACKEND

`e:\dreamspos\backend`

FRONTEND

`e:\dreamspos\angular`

2. Stack technique

BACKEND CORE

- **Python** 3.14
- **Django** 5.1.4
- **DRF** 3.15.2
- **SQLite** (dev) / **PostgreSQL** (prod)

SÉCURITÉ & PAIEMENT

- **SimpleJWT** 5.3.1 + Blacklist
- **pyotp** 2.9.0 (2FA TOTP)
- **Campay** (MTN MoMo / Orange Money)

OUTILS

- **drf-spectacular** 0.27.2 (OpenAPI)
- **django-modeltranslation** 0.18.11
- **Redis** (cache paramètres + token Campay)
- **python-slugify** 8.0.4

- Verrouillage compte (5 tentatives)

Commandes de démarrage

```
# Windows PowerShell
cd e:\dreamspos\backend
$env:DJANGO_SETTINGS_MODULE = "config.settings.local"
venv\Scripts\python.exe manage.py runserver 8000

# Swagger UI    →  http://127.0.0.1:8000/api/docs/
# Admin Django  →  http://127.0.0.1:8000/admin/
# Identifiants dev : admin@dreamspos.com / Admin1234!
```

Variables d'environnement (.env)

```
SECRET_KEY=...
DEBUG=True
ALLOWED_HOSTS=*
DATABASE_URL=sqlite:///db.sqlite3

# Campay (Mobile Money)
CAMPAY_APP_USERNAME=...
CAMPAY_APP_PASSWORD=...
CAMPAY_BASE_URL=https://demo.campay.net/api/   # sandbox
CAMPAY_WEBHOOK_URL=https://votre-domaine.com/api/v1/online/webhooks/campay/
CAMPAY_REDIRECT_URL=https://votre-app.com/booking-result
CAMPAY_WEBHOOK_SECRET=...   # optionnel — vérification signature HMAC
```

3. Architecture backend

Pattern Services / Selectors

Request HTTP → URLs → Views (thin) → Serializers → Services / Selectors → Models → DB

SELECTORS (LECTURES)

QuerySets, filtres, annotations, agrégations. Aucune écriture en base.

SERVICES (ÉCRITURES)

Logique métier, `@transaction.atomic`, `select_for_update()` pour les soldes. Lève `ApplicationError`.

Multi-tenancy — Shared Schema

Tous les modèles métier portent un champ `company_id` (FK → `system_config.Company`). Les données d'une société ne sont jamais visibles par une autre.

Pattern **company=None** pour les données partagées (ex: `VehicleCategory`, `FareClass`, `MaintenanceType`) — accessibles à toutes les sociétés via `Q(company_id=X) | Q(company__isnull=True)`.

BaseModel

- **UUID** pk (non devinable)
- **created_at / updated_at** auto
- **Soft delete** : `is_deleted`, `deleted_at`, `soft_delete(user)`

- **Audit** : `created_by`, `updated_by`, `deleted_by`
- `OdometerLog` et `ParcelEvent` héritent de `models.Model` (logs immuables — pas de soft-delete)

4. Structure du projet

```
e:\dreamspos\backend\
├── config/
│   ├── settings/base.py        ← Settings partagés
│   ├── settings/local.py       ← Dev (DEBUG, SQLite)
│   └── urls.py
├── api/v1/router.py            ← Agrégateur de toutes les URLs
├── apps/
│   ├── core/                   ✅ BaseModel, permissions, exceptions, pagination
│   ├── authentication/         ✅ User, JWT, 2FA, sessions — 15 ep
│   ├── system_config/          ✅ Company, Agencies, Parameters — 26 ep
│   ├── admin_rbac/             ✅ Roles, Profiles, Permissions — 25 ep
│   ├── users/                  ✅ CRUD utilisateurs institution — 12 ep
│   ├── hr/                     ✅ Employés, contrats, congés — 40 ep
│   ├── fleet/                  ✅ Véhicules, maintenance, accidents — 43 ep
│   ├── transport_config/       ✅ Villes, terminaux, routes, tarifs — 18 ep
│   ├── voyage/                 ✅ Voyages, équipage, embarquement — 11 ep
│   ├── ticketing/              ✅ Clients, réservations, billets QR, bagages — 18 ep
│   ├── parcel/                 ✅ Colis, tracking, paiement — 16 ep
│   ├── treasury/               ✅ Caisses, banques, sessions, transactions — 19 ep
│   └── online_booking/         ✅ API publique + Campay MoMo — 8 ep
├── docs/
│   └── dreamspos_erp_backend.html
├── locale/fr|en|ar/LC_MESSAGES/django.po
```

```
├── .env
└── manage.py
```

5. Module Authentication ✅ COMPLET

Modèles

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`User`	Utilisateur étendu (AbstractUser)	UUID pk, company, default_agency, agencies (M2M), phone, avatar, job_title, language_code, timezone, is_institution_admin, is_2fa_enabled, totp_secret, must_change_password, failed_login_attempts, locked_until
`UserAgency`	Jointure User ↔ Agency	user, agency, is_primary
`UserSession`	Sessions JWT actives	user, jti, ip_address, user_agent, is_active, expires_at
`LoginAttempt`	Journal de connexion	email, ip_address, success, failure_reason, attempted_at
`PasswordResetToken`	Token reset mot de passe	user, token, expires_at (TTL 1h), usage unique

Endpoints — `/api/v1/auth/`

M	URL	ACCÈS	DESCRIPTION
POST	login/	Public	Connexion email + mot de passe → access + refresh ou requires_2fa
POST	login/2fa/	Public	Validation code TOTP (étape 2)
POST	refresh/	Public	Renouvellement access token
POST	logout/	JWT	Révocation refresh token courant
POST	logout-all/	JWT	Révocation de toutes les sessions
GET	me/	JWT	Profil complet utilisateur connecté
PATCH	me/	JWT	Mise à jour profil (phone, avatar, langue…)
GET	sessions/	JWT	Liste des sessions actives
DELETE	sessions/<jti>/	JWT	Révoquer une session spécifique
POST	change-password/	JWT	Changement mot de passe + révoque toutes sessions
POST	password-reset/	Public	Demande de réinitialisation par email
POST	password-reset/confirm/	Public	Réinitialisation via token email
POST	2fa/setup/	JWT	Génère secret TOTP + URI QR code
POST	2fa/verify/	JWT	Vérifie code TOTP + active 2FA
POST	2fa/disable/	JWT	Désactive 2FA (mot de passe requis)

6. Module System Config ✅ COMPLET

Packages commerciaux, institutions (tenants), agences, paramètres système (Redis) et feature flags.

Modèles

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`Package`	Offre commerciale	code, tier, max_users, max_agencies, price_monthly, price_yearly, has_api_access…
`Company`	Institution / Tenant	name, slug (auto), logo, legal_form, rccm, tax_id, currency, package, status (Draft/Trial/Active/Suspended/Cancelled)
`Subscription`	Souscription active	company, package, billing_cycle, started_at, expires_at, trial_ends_at
`Agency`	Agence / Filiale	company, parent (hiérarchie illimitée), name, code, agency_type, manager
`Parameter`	Paramètre clé/valeur	key, value, data_type, scope, company. Caché Redis 5 min.
`FeatureFlag`	Activation modules	key, company, is_enabled. Caché Redis 5 min.

Endpoints — `/api/v1/system/`

M	URL	ACCÈS	DESCRIPTION
GET	packages/	Public	Liste des packages
POST	packages/admin/	SuperAdmin	Créer un package
PATCH	packages/admin/<id>/	SuperAdmin	Modifier un package
GET	companies/	SuperAdmin	Liste toutes les institutions
POST	companies/	SuperAdmin	Créer institution (slug auto, trial auto)
PATCH	companies/<id>/	SuperAdmin	Modifier institution
POST	companies/<id>/suspend/	SuperAdmin	Suspendre avec raison
POST	companies/<id>/activate/	SuperAdmin	Réactiver
POST	companies/<id>/cancel/	SuperAdmin	Résilier
GET	companies/<id>/stats/	SuperAdmin	Statistiques nb users, agences, limites
POST	companies/<id>/change-package/	SuperAdmin	Changer de package
GET	agencies/	InstitAdmin	Liste agences
POST	agencies/	InstitAdmin	Créer agence (limite package vérifiée)
GET	agencies/tree/	InstitAdmin	Arborescence agences
PATCH	agencies/<id>/	InstitAdmin	Modifier agence
DELETE	agencies/<id>/	InstitAdmin	Supprimer (bloqué si enfants actifs)
GET	parameters/	SuperAdmin	Liste paramètres
POST	parameters/bulk/	SuperAdmin	Upsert en masse
GET	feature-flags/	SuperAdmin	Liste des feature flags
POST	feature-flags/<id>/toggle/	SuperAdmin	Inverser état du flag

7. Module Admin RBAC ✅ COMPLET

Hiérarchie : **Modules → Menus → Permissions → Rôles → Profils → Utilisateurs**

Modèles

MODÈLE	DESCRIPTION
`AppModule`	Module fonctionnel (code, icon, order, parent, is_core)
`Menu`	Entrée Angular router (module, parent, route, menu_type, order)
`Permission`	Droit granulaire (module, code ex: hr.employee.view, category)
`Role`	Ensemble de permissions (company=null=système, is_system)
`Profile`	Template de rôles réutilisable (is_default, is_system)
`UserRole`	Rôle direct user→role avec expiration optionnelle et scope agence
`UserProfile`	Profil assigné à un utilisateur (1 seul par user/company)

8. Module Users ✅ COMPLET

CRUD utilisateurs par institution. Invitation avec `temp_password` auto-généré, activation/désactivation, gestion des agences.

Endpoints — `/api/v1/users/`

M	URL	ACCÈS	DESCRIPTION
GET	users/	InstitAdmin	Liste paginée (search, is_active, agency_id)
POST	users/	InstitAdmin	Créer — retourne temp_password, vérifie limite package
GET	users/stats/	InstitAdmin	Statistiques : total, actifs, verrouillés, must_change
GET	users/<id>/	InstitAdmin	Profil complet + agences
PATCH	users/<id>/	InstitAdmin	Modifier profil
DELETE	users/<id>/	InstitAdmin	Désactiver + révoque sessions JWT
POST	users/<id>/activate/	InstitAdmin	Réactiver
POST	users/<id>/unlock/	InstitAdmin	Déverrouiller compte bloqué
POST	users/<id>/reset-password/	InstitAdmin	Reset mot de passe par admin
GET	users/<id>/agencies/	InstitAdmin	Agences de l'utilisateur
PATCH	users/<id>/agencies/	InstitAdmin	Remplacer liste agences

9. Module RH (Human Resources) ✅ COMPLET

9 modèles, 40 endpoints. Grades, types de contrat/congé, départements (arbre), postes, employés (numéro `EMP-YEAR-SEQ` auto), contrats, soldes et workflow de congés.

Modèles (9)

CONFIGURATION

- **Grade** — fourchette salariale
- **ContractType** — CDI/CDD/Alternance (system ou company)
- **LeaveType** — types de congé avec règles de solde
- **Department** — arbre (parent auto-ref)
- **JobPosition** — poste rattaché à département + grade

GESTION

- **Employee** — dossier complet, numéro auto, FK user optionnel
- **Contract** — contrat courant, salaires, primes
- **LeaveBalance** — solde annuel par (employé, type, année)
- **LeaveRequest** — workflow PENDING→APPROVED/REJECTED/CANCELLED

Règles métier clés

NUMÉRO EMPLOYÉ

`EMP-{YEAR}-{SEQ:04d}` généré avec `select_for_update()` pour éviter les doublons concurrent.

CONGÉS

Comptage jours ouvrables (lun–ven). Solde auto-créé (`get_or_create`). Débit à l'approbation, recréditation à l'annulation.

10. Module Fleet (Gestion de flotte) ✅ COMPLET

Gestion complète de la flotte : véhicules, conducteurs externes, carburant, maintenance préventive, documents, assurances, missions et accidents. **43 endpoints**.

Modèles (13)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`VehicleBrand`	Marque globale (no company FK)	name, country_of_origin
`VehicleModel`	Modèle de véhicule	brand, name, vehicle_type, fuel_type, seats_capacity
`ExternalDriver`	Conducteur non-utilisateur système	company, first_name, last_name, phone, license_category (A/B/C/D/E), license_expiry
`Vehicle`	Véhicule de la flotte	company, agency, model, license_plate, year, status (AVAILABLE/ON_MISSION/IN_MAINTENANCE/OUT_OF_SERVICE), internal_driver, external_driver, current_odometer
`OdometerLog`	Log odomètre immuable (hérite Model)	vehicle, value_km, source (MANUAL/FUEL/MAINTENANCE/ASSIGNMENT), recorded_by
`FuelLog`	Ravitaillement carburant	vehicle, liters, amount, station, full_tank, odometer_km (auto-update)
`MaintenanceType`	Type de maintenance (company=null=global)	name, interval_km, interval_days (alertes prédictives)
`MaintenanceOrder`	Bon de maintenance	vehicle, type, status, odometer_at_start, cost, next_due_km, next_due_date
`Insurance`	Assurance véhicule	vehicle, policy_number, insurer, premium, start_date, end_date, `days_until_expiry` property
`VehicleDocument`	Documents réglementaires	vehicle, doc_type (CARTE_GRISE/VISITE_TECHNIQUE/...), expiry_date, `days_until_expiry` property
`Assignment`	Mission véhicule	vehicle, driver interne/externe, purpose, start/end, odometer, distance_km property
`Accident`	Accident lié à une mission	vehicle, assignment, severity (MINOR/MODERATE/MAJOR), status (REPORTED→INVESTIGATED→CLOSED), driver_name property

Règles métier clés

STATUT VÉHICULE AUTO-GÉRÉ

- `start_assignment()` → **ON_MISSION**
- `complete_assignment()` → **AVAILABLE** + màj odomètre
- `start_maintenance()` → **IN_MAINTENANCE**
- `complete_maintenance()` → **AVAILABLE** + calcule next_due
- Suppression bloquée si ON_MISSION ou IN_MAINTENANCE

ALERTES FLOTTE

- `get_fleet_alerts()` retourne assurances et documents expirant dans 30 j
- `get_vehicle_stats()` calcule TCO (coût total de possession)
- Maintenance prédictive par intervalle km + jours
- Intégration voyage : départ → ON_MISSION, arrivée → AVAILABLE

Endpoints — `/api/v1/fleet/` (43)

M	URL	DESCRIPTION
GET / POST	brands/ | brands/<id>/	CRUD marques (global)
GET / POST	models/ | models/<id>/	CRUD modèles de véhicule
GET / POST	external-drivers/ | external-drivers/<id>/	CRUD conducteurs externes
GET	vehicles/alerts/	Alertes flotte (expirations, maintenance)
GET	vehicles/stats/	Statistiques flotte (TCO, disponibilité)
GET / POST	vehicles/ | vehicles/<id>/	CRUD véhicules
POST	vehicles/<id>/odometer/	Mettre à jour odomètre
GET / POST	vehicles/<id>/fuel-logs/	Logs carburant d'un véhicule
GET / POST	vehicles/<id>/maintenance/	Maintenance d'un véhicule
POST	vehicles/<id>/maintenance/<mid>/start/	Démarrer maintenance → IN_MAINTENANCE
POST	vehicles/<id>/maintenance/<mid>/complete/	Terminer + calculer next_due
GET / POST	vehicles/<id>/insurance/ | /<iid>/	CRUD assurances
GET / POST	vehicles/<id>/documents/ | /<did>/	CRUD documents réglementaires
GET / POST	vehicles/<id>/assignments/	Missions d'un véhicule
POST	assignments/<id>/start/ | complete/	Démarrer / Terminer mission
GET / POST	vehicles/<id>/accidents/	Accidents d'un véhicule
POST	accidents/<id>/status/	Changer statut accident (REPORTED→INVESTIGATED→CLOSED)
GET	maintenance-types/ | fuel-logs/ | assignments/ | accidents/	Vues globales (toute la flotte)

11. Module Transport Config ✅ COMPLET

Données de référence du système transport : villes, terminaux, routes, catégories de véhicules, classes tarifaires et tarifs. **18 endpoints**.

Modèles (7)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`City`	Ville (global — pas de company FK)	name, code (unique), country
`Terminal`	Terminal/Gare d'une société	company, city, name, address, latitude, longitude. unique_together (company, city, name)
`Route`	Itinéraire entre 2 terminaux	company, from_terminal, to_terminal, distance_km, duration_minutes, name (auto-généré). unique_together (company, from_terminal, to_terminal)
`RouteStop`	Arrêts intermédiaires	route, terminal, order (unique par route), duration_from_start_min
`VehicleCategory`	Catégorie de bus/car (company=null=global)	name, total_seats, seat_rows, seat_cols, aisle_after_col, has_back_row, free_seating, has_wifi/has_ac/has_tv/has_usb/has_snack. `amenities_list` property
`FareClass`	Classe tarifaire (company=null=global)	name, description
`Fare`	Tarif (route × catégorie × classe × type passager)	route, vehicle_category, fare_class, passenger_type (ADULT/CHILD/STUDENT/SENIOR), price. unique_together (route, vehicle_category, fare_class, passenger_type)

Endpoints — `/api/v1/transport/` (18)

M	URL	DESCRIPTION
GET / POST	cities/ | cities/<id>/	CRUD villes (global)
GET / POST	terminals/ | terminals/<id>/	CRUD terminaux
GET / POST	routes/ | routes/<id>/	CRUD routes
POST	routes/<id>/stops/	Ajouter arrêt intermédiaire (ordre unique, terminal pas déjà dans la route)
DELETE	routes/<id>/stops/<sid>/	Supprimer arrêt
GET / POST	vehicle-categories/ | /<id>/	CRUD catégories (global ou company)
GET / POST	fare-classes/ | /<id>/	CRUD classes tarifaires (global)
GET / POST	fares/ | fares/<id>/	CRUD tarifs (route × catégorie × classe × passager)

12. Module Voyage (Gestion des départs) ✅ COMPLET

Création et suivi des voyages, embarquement des passagers, gestion de l'équipage. **11 endpoints**.

Modèles (2)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`Trip`	Voyage programmé — unité centrale du transport	company, reference (auto: TRIP-YYYYMMDD-NNN), route, departure_datetime, estimated/actual_arrival/departure, vehicle (optionnel), vehicle_category (requis), fare_class, total_seats, free_seating, status (SCHEDULED/BOARDING/DEPARTED/ARRIVED/CANCELLED). `from_city`, `to_city` properties. `booked_seats` annoté.
`TripCrew`	Membre d'équipage affecté à un voyage	trip, user, role (DRIVER/CONDUCTOR/SUPERVISOR/OTHER), notes. unique_together (trip, user)

Cycle de vie d'un voyage

```
SCHEDULED → BOARDING → DEPARTED → ARRIVED
                ↓                    ↘
            CANCELLED          CANCELLED bloqué
```

Intégration Fleet : `depart_trip()` → Vehicle.status = ON_MISSION | `arrive_trip()` ou `cancel_trip()` → Vehicle.status = AVAILABLE

Endpoints — `/api/v1/voyage/` (11)

M	URL	DESCRIPTION
GET	trips/stats/	Tableau de bord (upcoming, boarding, in_transit, arrived_today)
GET	trips/search/	Recherche publique (from_city, to_city, date)
GET / POST	trips/	Liste paginée + création
GET / PATCH / DELETE	trips/<id>/	Détail, mise à jour (SCHEDULED seulement), suppression
POST	trips/<id>/board/	SCHEDULED → BOARDING (ouverture embarquement)
POST	trips/<id>/depart/	→ DEPARTED + vehicle ON_MISSION
POST	trips/<id>/arrive/	→ ARRIVED + vehicle AVAILABLE
POST	trips/<id>/cancel/	Annulation
GET / POST	trips/<id>/crew/	Équipage d'un voyage
GET / DELETE	trips/<id>/crew/<cid>/	Membre d'équipage

13. Module Ticketing ✅ COMPLET

Gestion des clients, réservations, billets avec QR code, et bagages. **18 endpoints**.

Modèles (4)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`Customer`	Passager non-utilisateur système	company, first_name, last_name, phone, email, id_type (CNI/PASSPORT/...), id_number, date_of_birth. `full_name` property
`Booking`	Réservation sur un voyage	company, reference (auto: BKG-YYYYMMDD-NNNN), trip (related_name="bookings"), customer, passenger_type (ADULT/CHILD/STUDENT/SENIOR), seat_number, fare, amount_paid, status (PENDING/CONFIRMED/BOARDED/CANCELLED/NO_SHOW)
`Ticket`	Billet numérique scannable	booking (OneToOne), ticket_number (auto: TKT-...), qr_code (UUID, unique), issued_at, issued_by, is_used, used_at
`Luggage`	Bagage lié à une réservation	booking, tag_number (auto: LGG-...), description, weight_kg, status (CHECKED_IN/LOADED/DELIVERED/LOST)

Règles métier clés

CRÉATION RÉSERVATION

- Vérifie places disponibles via annotation SQL en temps réel
- Vérifie unicité du siège pour voyages à places assignées
- Crée automatiquement le Ticket avec QR code UUID unique
- `booked_seats` sur Trip = Count(CONFIRMED + BOARDED)

EMBARQUEMENT PAR SCAN

- `board_by_qr(qr_code)` → CONFIRMED → BOARDED
- Ticket marqué `is_used=True` + `used_at`
- Erreur si ticket déjà utilisé
- Bagages : CHECKED_IN → LOADED → DELIVERED (transitions validées)

Endpoints — `/api/v1/ticketing/` (18)

M	URL	DESCRIPTION
GET	bookings/stats/	Stats (total, confirmed, boarded, revenue) — filtrable par trip_id
POST	bookings/scan/	Embarquement par scan QR code
GET / POST	customers/ | /<id>/	CRUD clients
GET / POST	bookings/ | /<id>/	Liste + création réservation
POST	bookings/<id>/confirm/	PENDING → CONFIRMED + crée Ticket
POST	bookings/<id>/board/	CONFIRMED → BOARDED + marque ticket is_used
POST	bookings/<id>/cancel/	Annuler réservation
POST	bookings/<id>/no-show/	CONFIRMED → NO_SHOW
GET	trips/<id>/bookings/	Toutes réservations d'un voyage (vue embarquement)
GET / POST	bookings/<id>/luggage/ | /<lid>/	CRUD bagages
POST	bookings/<id>/luggage/<lid>/status/	Changer statut bagage

14. Module Parcel (Colis) ✅ COMPLET

Gestion du cycle de vie des colis expédiés par bus. Tracking complet + paiement COD. **16 endpoints**.

Modèles (2)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`Parcel`	Colis expédié entre deux terminaux	company, reference (auto: PCL-YYYYMMDD-NNNN), from_terminal, to_terminal, trip (optionnel), sender_name/phone, recipient_name/phone/address, description, weight_kg, quantity, declared_value, shipping_amount, payment_status (UNPAID/PAID/COD), payment_mode (CASH/MOBILE_MONEY/TRANSFER), paid_at, paid_by, status (REGISTERED/SHIPPED/RECEIVED/DELIVERED/RETURNED/LOST)
`ParcelEvent`	Log de tracking immuable (hérite models.Model)	parcel, status, location, note, created_by, created_at

Cycle de vie

```
REGISTERED → SHIPPED → RECEIVED → DELIVERED
                                ↘          ↘
                  RETURNED    RETURNED    (tout état) → LOST
```

COD : payment_status=COD → encaissement automatique à la livraison (`deliver_parcel()` appelle `record_payment()`).
ParcelEvent : chaque transition crée une entrée immuable de tracking.

Endpoints — `/api/v1/parcels/` (16)

M	URL	DESCRIPTION
GET	stats/	Stats par statut + revenus
GET	trips/<id>/parcels/	Colis d'un voyage
GET / POST	(vide) | <id>/	Liste (search, status, payment_status, date) + CRUD
POST	<id>/assign-trip/	Affecter à un voyage
POST	<id>/ship/	REGISTERED → SHIPPED
POST	<id>/receive/	SHIPPED → RECEIVED (terminal destination)
POST	<id>/deliver/	RECEIVED → DELIVERED (remet au destinataire + COD auto)
POST	<id>/return/	→ RETURNED
POST	<id>/lost/	→ LOST
POST	<id>/payment/	Enregistrer paiement (CASH/MOBILE_MONEY/TRANSFER)
GET	<id>/events/	Historique de tracking (ParcelEvent)

15. Module Treasury (Trésorerie) ✅ COMPLET

Gestion des caisses physiques, comptes bancaires, sessions de caisse journalières, transactions financières et virements internes. **19 endpoints**.

Modèles (5)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`CashRegister`	Caisse physique par terminal	company, name, terminal (optionnel), responsible (FK User), current_balance (mis à jour en temps réel), is_active
`BankAccount`	Compte bancaire société	company, bank_name, account_name, account_number, current_balance, is_active
`CashSession`	Session quotidienne d'une caisse	company, cash_register, opened_by, closed_by, opening_balance, closing_balance, expected_closing, difference, opened_at, closed_at, status (OPEN/CLOSED)
`Transaction`	Mouvement financier (revenu ou dépense)	company, reference (auto: TXN-...), transaction_type (INCOME/EXPENSE), category (TICKET_SALES/PARCEL_SHIPPING/CHARTER/FUEL/MAINTENANCE/...), account_type (CASH/BANK), cash_session, bank_account, amount, payment_mode (CASH/MOBILE_MONEY/TRANSFER/CHECK), transaction_date, trip/booking/parcel (FK optionnels)
`Transfer`	Virement interne entre comptes	company, reference (auto: TRF-...), from_cash_register/from_bank_account, to_cash_register/to_bank_account, amount, description, transfer_date, completed_by

Règles métier clés

SOLDES EN TEMPS RÉEL

- `create_transaction()` met à jour le solde caisse ou banque atomiquement avec `select_for_update()`
- `delete_transaction()` inverse l'impact sur le solde (bloqué si session fermée)

VIREMENTS

- Validation solvabilité de la source avant débit
- Opération atomique : débit source + crédit destination en un seul `@transaction.atomic`
- Supporte caisse→caisse, caisse→banque, banque→caisse, banque→banque

- `close_session()` calcule l'écart : `closing_balance - expected_closing`

Endpoints — `/api/v1/treasury/` (19)

M	URL	DESCRIPTION
GET	stats/	Soldes total cash+banque, revenus/dépenses du jour, sessions ouvertes
GET / POST	registers/ | /<id>/	CRUD caisses (?active=true)
GET / POST	banks/ | /<id>/	CRUD comptes bancaires
GET	sessions/	Liste sessions (filter: register_id, status, date_from, date_to)
POST	sessions/open/	Ouvrir session (bloqué si session déjà ouverte sur la caisse)
GET	sessions/<id>/	Détail session
POST	sessions/<id>/close/	Fermer + calculer écart (closing_balance - expected)
GET / POST	transactions/ | /<id>/	Transactions (filter: type, category, account_type, session, bank, date, trip)
GET / POST	transfers/ | /<id>/	Virements internes

16. Module Online Booking (API Publique)

✅ COMPLET

API sans authentification pour les clients finaux : recherche de voyages, sélection de siège, paiement Mobile Money via Campay (MTN MoMo / Orange Money), callback webhook. **8 endpoints (AllowAny)**.

Modèle (1)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`OnlineBookingRequest`	Demande de réservation publique en attente de paiement	company, trip, passenger_name/phone/email, passenger_type, seat_number, fare, amount, payment_method (MTN_MOMO/ORANGE_MONEY), payment_phone, payment_reference (Campay ref), payment_operator_ref, status (PENDING/PAID/FAILED/EXPIRED), expires_at (created_at + 15 min), booking (FK créé sur succès paiement)

Flux complet de réservation en ligne

```
[1] Client → GET /online/<company_id>/trips/?from_city=X&to_city=Y&date=Z [2] Client → GET /online/<company_id>/trips/<id>/seats/ # plan de siège [3] Client → POST /online/<company_id>/trips/<id>/book/ → OnlineBookingRequest(PENDING) créé → CampayClient.collect() appelé ← { request_id, payment_reference, expires_at } [4] Campay → POST /online/webhooks/campay/ status=SUCCESSFUL → confirm_payment() → Customer.get_or_create(phone) → Booking + Ticket créés automatiquement status=FAILED → fail_payment() [5] Client → GET /online/requests/<request_id>/ → sync_payment_status() # polling Campay si PENDING ← { status: PAID, ticket: { ticket_number, qr_code } }
```

CampayClient (apps/online_booking/campay/client.py)

AUTHENTIFICATION

- Token obtenu via `POST /token/` (username + password)
- Mis en cache Redis pendant **23 heures** (clé: `campay_access_token`)
- Client HTTP natif Python (`urllib` — aucune dépendance externe)

SÉCURITÉ WEBHOOK

- Vérification HMAC-SHA256 si `CAMPAY_WEBHOOK_SECRET` configuré
- Header : `X-Campay-Signature`
- En-tête manquant → 403 si secret configuré
- Toujours retourner HTTP 200 pour ACK (évite les retries Campay)

Endpoints — `/api/v1/online/` (8 — AllowAny)

M	URL	DESCRIPTION
POST	webhooks/campay/	Callback Campay — confirmation ou échec paiement
GET	requests/<request_id>/	Statut réservation — UUID est le token d'accès
GET	<company_id>/trips/	Recherche voyages disponibles (from_city, to_city, date)
GET	<company_id>/trips/<id>/	Détail voyage (places disponibles, catégorie, amenities)
GET	<company_id>/trips/<id>/seats/	Plan de siège (rows, cols, aisle, sièges pris)
POST	<company_id>/trips/<id>/book/	Initier réservation → paiement Campay

17. Récapitulatif des endpoints

15 Auth /api/v1/auth/	**26** System /api/v1/system/	**25** RBAC /api/v1/rbac/	**11** Users /api/v1/users/
40 RH /api/v1/hr/	**43** Fleet /api/v1/fleet/	**18** Transport /api/v1/transport/	**11** Voyage /api/v1/voyage/
18 Ticketing /api/v1/ticketing/	**16** Parcel /api/v1/parcels/	**19** Treasury /api/v1/treasury/	**8** Online Booking /api/v1/online/
41 Sync Sage 100 /api/v1/scig/			

291

Endpoints API au total

13 modules — 46 modèles Django — 15 migrations

TOUTES LES ROUTES — `/API/V1/`

PRÉFIXE	MODULE	ACCÈS
`auth/`	Authentification (JWT, 2FA, sessions)	Public + JWT
`system/`	Companies, Agencies, Parameters, FeatureFlags	SuperAdmin + InstitAdmin
`rbac/`	Modules, Menus, Permissions, Roles, Profiles	SuperAdmin + InstitAdmin
`users/`	Gestion utilisateurs par institution	InstitAdmin
`hr/`	Employés, contrats, congés	InstitAdmin
`fleet/`	Véhicules, maintenance, accidents	JWT
`transport/`	Villes, terminaux, routes, tarifs	JWT
`voyage/`	Voyages, embarquement, équipage	JWT
`ticketing/`	Clients, réservations, billets QR, bagages	JWT
`parcels/`	Colis, tracking, paiement	JWT
`treasury/`	Caisses, banques, sessions, transactions	JWT
`online/`	API publique + Campay webhook	**Public (AllowAny)**
`scig/`	Sync Sage 100 — inventaire mobile, transferts, articles, connexion	JWT + FeatureFlag

18. Roadmap des modules

Verticale Transport & Logistique — Complète ✅

✅ Core (`apps.core`)

BaseModel, Address, Currency, UoM, Attachment, TenantMiddleware, permissions (IsSuperAdmin, IsInstitutionAdmin, IsAuthenticated), exceptions, responses, pagination

✅ Authentication (`apps.authentication`)

User étendu (AbstractUser), JWT custom claims, 2FA TOTP (RFC 6238), verrouillage compte, sessions tracées, reset mot de passe — 15 endpoints

✅ System Config (`apps.system_config`)

Packages commerciaux, Companies/Institutions (tenants), Subscriptions, Agencies (hiérarchie illimitée), Parameters (Redis), FeatureFlags (Redis) — 26 endpoints

✅ Admin RBAC (`apps.admin_rbac`)

AppModules, Menus Angular router, Permissions granulaires, Rôles, Profils, UserRole (avec expiration), UserProfile — 25 endpoints

✅ Users (`apps.users`)

CRUD utilisateurs par institution, invitation (temp_password), activation/désactivation + révocation JWT, unlock, reset password admin, gestion agences — 11 endpoints

✅ RH (`apps.hr`)

Grades, types contrat/congé, départements (arbre), postes, employés (EMP-YEAR-SEQ auto), contrats, soldes de congé, workflow approbation congés — 40 endpoints

✅ Fleet (`apps.fleet`)

VehicleBrand, VehicleModel, ExternalDriver, Vehicle (statut auto-géré), OdometerLog, FuelLog, MaintenanceType (global), MaintenanceOrder (next_due auto), Insurance, VehicleDocument, Assignment, Accident — 43 endpoints

✅ Transport Config (`apps.transport_config`)

City (global), Terminal, Route + RouteStop, VehicleCategory (plan siège + amenities), FareClass (global), Fare (route × catégorie × classe × passager) — 18 endpoints

✅ Voyage (`apps.voyage`)

Trip (référence auto, booked_seats annoté, intégration fleet), TripCrew. Transitions SCHEDULED→BOARDING→DEPARTED→ARRIVED/CANCELLED — 11 endpoints

✅ Ticketing (`apps.ticketing`)

Customer, Booking (PENDING→CONFIRMED→BOARDED/CANCELLED/NO_SHOW), Ticket (QR code UUID auto), Luggage (CHECKED_IN→LOADED→DELIVERED). Embarquement par scan QR — 18 endpoints

✅ Parcel (`apps.parcel`)

Parcel (cycle REGISTERED→SHIPPED→RECEIVED→DELIVERED/RETURNED/LOST, COD auto), ParcelEvent (log tracking immuable). Transition atomiques, paiement intégré — 16 endpoints

✅ Treasury (`apps.treasury`)

CashRegister, BankAccount, CashSession (écart opening/closing), Transaction (solde en temps réel via select_for_update), Transfer (virement interne atomique) — 19 endpoints

✅ Online Booking (`apps.online_booking`)

OnlineBookingRequest, CampayClient (MTN MoMo / Orange Money, token Redis 23h), webhook Campay HMAC-SHA256. Flux : recherche → siège → book → paiement → ticket QR — 8 endpoints publics

✅ Sync SCIG — Sage 100 (`apps.sync_scig_sage`)

Intégration SQL Server directe (pyodbc) avec Sage 100 Gestion Commerciale. SageConnection par company (mot de passe Fernet), sync articles/variants/dépôts/stocks, inventaire mobile Zebra (zones Rouge→Bleu→Vert), transferts inter-dépôts, impression étiquettes, queue offline — 41 endpoints. Accès par FeatureFlag `sync_scig_sage.enabled` .

Prochaines verticales ERP

🔧 Finance OHADA (`apps.finance`)

Plan comptable OHADA, journaux comptables, écritures, rapports financiers (bilan, compte de résultat), TVA, rapprochement trésorerie

🔧 Inventaire / Achats / Ventes (`apps.inventory, apps.purchase, apps.sales`)

Articles, stocks, bons de commande, factures, livraisons, fournisseurs, clients, avoirs

🔧 POS — Point de Vente (`apps.pos`)

Sessions de vente, tickets de caisse, modes de paiement multiples, clôture journalière, reçus

🔧 CRM (`apps.crm`)

Pipeline commercial, contacts, opportunités, activités, relances, rapports commerciaux

19. Module Sync SCIG — Sage 100 ✅ COMPLET

Intégration directe SQL Server (pyodbc) avec **Sage 100 Gestion Commerciale**. Chaque société a sa propre connexion Sage stockée de façon sécurisée. Le module gère la synchronisation des référentiels (articles, dépôts, stocks) et les opérations terrain sur terminaux Zebra (inventaires, transferts, étiquettes). **41 endpoints**.

Architecture d'intégration

Sage 100 SQL Server → SageClient (pyodbc) → mapper.py → Django ORM (PostgreSQL) → API REST → Terminaux Zebra

Modèles (14)

MODÈLE	DESCRIPTION	CHAMPS CLÉS
`SageConnection`	Paramètres SQL Sage par company (1:1)	company (OneToOne), server, port, database, username, password_encrypted (Fernet), last_test_success
`Article`	Article Sage (F_ARTICLE)	sage_reference, designation, barcode, family, sub_family, has_size_variant, has_color_variant
`ArticleVariant`	Variante (F_ARTGAMME — taille × couleur)	article, sage_ref_variant, size, color, barcode, price_normal, price_sale
`Depot`	Dépôt/magasin (F_DEPOT)	company, sage_depot_no, name, is_active
`StockLevel`	Stock par (variante, dépôt) (F_ARTSTOCK)	depot, variant, stock_real, stock_available, stock_transit, synced_at
`InventorySession`	Session d'inventaire physique	company, name, depot, status (OPEN/CLOSED), opened_at, closed_at, submitted_to_sage_at
`InventoryZone`	Zone de comptage (terminal Zebra)	session, name, status (ROUGE/BLEU/VERT), locked_by_device, validated_by
`InventoryLine`	Ligne comptée (variante × zone)	zone, variant, qty_theoretical, qty_counted, is_manual, last_scanned_at
`StockTransfer`	Transfert inter-dépôts	company, reference (TRF-DATE-SEQ), source_depot, destination_depot, status
`StockTransferLine`	Ligne de transfert	transfer, variant, qty_planned, qty_shipped, qty_received
`LabelCampaign`	Campagne d'étiquetage promotionnel	company, name, discount_percent, starts_at, ends_at
`LabelPrintJob`	Job d'impression étiquettes (Zebra)	campaign, print_type (NORMAL/PROMO), print_data (JSON), device_id, total_labels
`SyncLog`	Historique de synchronisation Sage	sync_type (DEPOTS/ARTICLES/STOCK/FULL), status, records_read/created/updated/failed
`SyncQueue`	File d'attente opérations offline Zebra	device_operation_id (unique, idempotent), operation_type, device_timestamp, payload (JSON), status

Chiffrement des mots de passe Sage

`sage/encryption.py` — Fernet (AES-128-CBC) dérivé de `SECRET_KEY` via SHA-256. Aucune clé externe à gérer.

```
# Transparent via property — jamais exposé en clair dans l'API
conn.password = "MonMotDePasse"          # encrypt automatique
plaintext = conn.password                # decrypt automatique
conn_str  = conn.build_connection_string()  # prêt pour pyodbc
```

SageClient — connexion multi-tenant

```
# Dans services.py — helper central
def _get_sage_client(company) -> SageClient:
    conn = SageConnection.objects.filter(company=company, is_active=True).first()
    if not conn:
        raise ApplicationError("Aucune connexion Sage configurée.")
    return SageClient(conn)   # SageClient(connection=conn)

# Fallback dev (settings.SAGE_SQL_*)
client = SageClient()         # sans argument = legacy/dev
```

Workflow inventaire (terminaux Zebra)

ROUGE — IN_PROGRESS → BLEU — SUBMITTED → ORANGE — RECHECK →

VERT — VALIDATED → Push Sage 100

Zone ouverte par le terminal (lock `device_id`) → scan articles → soumission → validation manager → push F_ARTSTOCK. Submit bloqué tant que toutes les zones ne sont pas VALIDATED.

Règles métier clés

- **1 SageConnection par company** (`OneToOneField`) — upsert via `upsert_sage_connection()`
- **Accès conditionné au FeatureFlag** `sync_scig_sage.enabled` via `IsScigSageEnabled`
- **Queue offline idempotente** — `device_operation_id` unique, doublons silencieusement ignorés
- **Snapshot stock théorique** à l'ouverture de zone — comparaison physique vs Sage
- **Transfert** : vérification stock disponible avant ajout de ligne (évite sur-planification)
- **Push Sage** : update atomique `F_ARTSTOCK.AS_StockReel` avec rollback pyodbc

Endpoints — `/api/v1/scig/` (41)

MÉTHODE	URL	DESCRIPTION
GET	dashboard/	KPIs manager (sessions ouvertes, transferts en cours, stocks critiques)
GET POST	connection/	Lire / créer-mettre à jour la connexion Sage 100 (server, port, db, user, pwd)
POST	connection/test/	Test live de connexion SQL Server — met à jour last_test_success
POST	sync/trigger/	Déclencher sync Sage (DEPOTS / ARTICLES / STOCK / FULL)
GET	sync/logs/	Historique des synchronisations avec filtres
POST	sync/queue/push/	Pousser opérations offline depuis terminal Zebra (batch idempotent)
POST	sync/queue/flush/	Traiter la file d'attente offline (processed/skipped/failed)
GET	articles/	Liste articles avec recherche/filtre famille
GET	articles/<id>/	Détail article + variantes
GET	variants/search/?q=	Recherche variante par code-barres ou référence (mobile Zebra)
GET	depots/	Liste des dépôts actifs
GET	depots/<id>/stock/	Stock d'un dépôt (réel/dispo/transit)
GET	stock/critical/	Articles en rupture ou stock critique (seuil paramétrable)
GET POST	inventory/	Liste / créer session d'inventaire
GET	inventory/<id>/	Détail session + zones
POST	inventory/<id>/submit-sage/	Push ajustements vers Sage 100 + clôture session
GET	inventory/<id>/report/	Rapport comparatif Sage vs physique (toutes lignes)
GET POST	inventory/<id>/zones/	Liste / créer zone de comptage
POST	inventory/<sid>/zones/<zid>/open/	Ouvrir zone → ROUGE (lock terminal)
POST	inventory/<sid>/zones/<zid>/scan/	Scanner code-barres (incrémente qty_counted)

MÉTHODE	URL	DESCRIPTION
POST	inventory/<sid>/zones/<zid>/manual/	Saisie manuelle quantité (écrase qty_counted)
POST	inventory/<sid>/zones/<zid>/submit/	Soumettre zone → BLEU (mobile)
POST	inventory/<sid>/zones/<zid>/validate/	Valider zone → VERT (dashboard manager)
POST	inventory/<sid>/zones/<zid>/recheck/	Renvoyer en vérification → ORANGE
GET	inventory/<sid>/zones/<zid>/lines/	Lignes comptées avec écart Sage/physique
GET POST	transfers/	Liste / créer transfert inter-dépôts
GET	transfers/<id>/	Détail transfert + lignes
POST	transfers/<id>/lines/scan/	Ajouter article par scan (DRAFT — vérifie stock dispo)
POST	transfers/<id>/validate/	DRAFT → VALIDATED
POST	transfers/<id>/ship/	VALIDATED → SHIPPED (copie qty_planned → qty_shipped)
POST	transfers/<id>/receive/scan/	Scanner article à la réception (cumule qty_received)
POST	transfers/<id>/receive/close/	Finaliser réception → RECEIVED
GET POST	labels/campaigns/	Liste / créer campagne d'étiquetage
GET POST	labels/print/	Liste / créer job d'impression étiquettes (Zebra)

20. Conventions de développement

Règles absolues

1. **Jamais `auth.User` dans les ForeignKey.** Toujours `settings.AUTH_USER_MODEL` ou `"authentication.User"` .
2. **Jamais d'ORM dans les views.** Selectors pour les lectures, services pour les écritures.
3. **Tous les modèles métier héritent de `BaseModel`** (UUID, timestamps, soft-delete, audit). Exception : logs immuables (`OdometerLog` , `ParcelEvent`) → `models.Model` .
4. **Services Write : paramètres explicites.** Pas de `**kwargs → objects.create()` — risque de conflits FK résolues.
5. **FK optionnelles dans les WriteSerializers :** `required=False, allow_null=True, default=None` .
6. `select_for_update()` pour toute mise à jour de solde (CashRegister.current_balance, BankAccount.current_balance) → prévient les race conditions.
7. **Références croisées entre modules :** utiliser les strings FK (`"fleet.Vehicle"` , `"voyage.Trip"`) pour éviter les circular imports.

Pattern company=None (données globales partagées)

Certains modèles de référence sont partagés entre toutes les sociétés : `VehicleCategory` , `FareClass` , `MaintenanceType` .

```
# Dans les selectors — récupérer données system + company
qs = VehicleCategory.objects.filter(
    Q(company_id=company_id) | Q(company__isnull=True),
    is_deleted=False
)
```

Structure type d'une app

FICHIER / DOSSIER	RÔLE
`apps.py`	AppConfig avec `label` unique
`models/`	Un fichier par entité + `__init__.py` d'export
`selectors/`	Fonctions de lecture (QuerySets, filtres, annotations)
`services/`	Logique métier + `@transaction.atomic`
`serializers.py`	Read + Write séparés quand les structures diffèrent
`views.py`	APIView thin — validation + délégation seulement
`urls.py`	Routes incluses depuis `api/v1/router.py`
`admin.py`	Interface Django Admin
`migrations/`	Migrations Django générées

Permissions disponibles

CLASSE	CONDITION	USAGE
`IsSuperAdmin`	is_authenticated + is_staff + is_superuser	Administration plateforme (packages, companies)
`IsInstitutionAdmin`	is_superuser OU is_institution_admin	Gestion interne d'une institution
`IsAuthenticated`	is_authenticated	Tout utilisateur connecté (fleet, voyage, treasury…)
`AllowAny`	—	Endpoints publics (login, packages, online booking)

Références auto-générées

MODULE	FORMAT	EXEMPLE
`Fleet — Employee`	EMP-{YEAR}-{SEQ:04d}	EMP-2026-0042
`Voyage — Trip`	TRIP-{YYYYMMDD}-{SEQ:03d}	TRIP-20260615-001
`Ticketing — Booking`	BKG-{YYYYMMDD}-{SEQ:04d}	BKG-20260615-0001
`Ticketing — Ticket`	TKT-{YYYYMMDD}-{SEQ:04d}	TKT-20260615-0001
`Ticketing — Luggage`	LGG-{YYYYMMDD}-{SEQ:04d}	LGG-20260615-0001
`Parcel`	PCL-{YYYYMMDD}-{SEQ:04d}	PCL-20260615-0001
`Treasury — Transaction`	TXN-{YYYYMMDD}-{SEQ:04d}	TXN-20260615-0001
`Treasury — Transfer`	TRF-{YYYYMMDD}-{SEQ:04d}	TRF-20260615-0001

Internationalisation (i18n)

Toutes les chaînes utilisateur enveloppées dans `gettext_lazy as _` . Langues : **Français** (défaut), **English**, **العربية**.

```
from django.utils.translation import gettext_lazy as _
raise ApplicationError(_("No seats available on this trip."))
```

Interface Rosetta : `http://127.0.0.1:8000/rosetta/`

DreamsPOS ERP — Documentation Backend v2.0.0
Mis à jour Juin 2026 | Django 5.1.4 + DRF 3.15.2 | Verticale Transport & Logistique complète — 250 endpoints — 11 modules — 38 modèles